



Social Media Platform (Such as Reddit)



Project Overview

The **Reddit Clone MVP** is a full-stack web application that enables users to create communities, share posts, vote, and comment—built using **Java-based backend technologies** and a modern frontend.

This project focuses on delivering a **scalable, production-ready MVP within 2 weeks**.



Objectives

- Build a RESTful backend using Java
 - Implement secure authentication & authorization
 - Design relational database with proper entity mapping
 - Integrate frontend with backend APIs
 - Deliver a deployable full-stack application
-



Core Features (MVP Scope)

1. User Authentication

- Sign up / Login
- JWT-based authentication
- Secure API access

2. Communities (Subreddits)

- Create community
- Browse communities
- Community-specific pages

3. Posts

- Create posts (text/image/link)
- View posts by community
- Post detail page

4. Voting System

- Upvote / Downvote
- Maintain vote count

5. Comments

- Add comments to posts
- Display comments list

6. Sorting

- Sort by latest
- Sort by popularity

Use Cases

Actor	Use Case	Description
User	Register/Login	Authenticate to system
User	Create Community	Create subreddit
User	Browse Communities	Explore content
User	Create Post	Add post
User	Vote	Upvote/downvote
User	Comment	Participate in discussions
User	Sort	Filter posts

Tech Stack

◇ Backend (Java)

- **Framework:** Spring Boot
 - **Security:** Spring Security
 - **Authentication:** JWT (JSON Web Tokens)
 - **ORM:** Hibernate
 - **Build Tool:** Maven
-

◇ Frontend

- **Framework:** [Next.js](#) / React.js
 - **Styling:** Tailwind CSS
-

◇ Database

- **Database:** PostgreSQL
-

◇ Deployment

- Backend: Render / Railway
 - Frontend: Vercel
-

System Architecture

Frontend (Next.js)



REST API (Spring Boot)



Service Layer (Business Logic)



Repository Layer (JPA/Hibernate)



Database Design (JPA Entities)

@Entity

```
class User {  
    @Id @GeneratedValue  
    private Long id;  
    private String email;  
    private String username;  
    private String password;  
  
    @OneToMany(mappedBy = "author")  
    private List<Post> posts;  
}
```

@Entity

```
class Community {  
    @Id @GeneratedValue  
    private Long id;  
    private String name;  
    private String slug;  
  
    @OneToMany(mappedBy = "community")  
    private List<Post> posts;  
}
```

@Entity

```
class Post {  
    @Id @GeneratedValue  
    private Long id;  
    private String title;  
    private String content;  
    private String imageUrl;  
  
    @ManyToOne  
    private User author;  
  
    @ManyToOne  
    private Community community;  
  
    @OneToMany(mappedBy = "post")  
    private List<Comment> comments;
```

```
}

@Entity
class Comment {
    @Id @GeneratedValue
    private Long id;
    private String content;

    @ManyToOne
    private User author;

    @ManyToOne
    private Post post;
}

@Entity
class Vote {
    @Id @GeneratedValue
    private Long id;
    private String type; // UPVOTE / DOWNVOTE

    @ManyToOne
    private User user;

    @ManyToOne
    private Post post;
}
```

API Design (Sample Endpoints)

Auth APIs

POST /api/auth/signup
POST /api/auth/login

Community APIs

POST /api/communities
GET /api/communities
GET /api/communities/{slug}

Post APIs

POST /api/posts

GET /api/posts/{id}
GET /api/communities/{slug}/posts

Vote APIs

POST /api/votes

Comment APIs

POST /api/comments
GET /api/posts/{id}/comments



Development Plan (2 Weeks)



Week 1 – Backend Core

Day 1 – Setup

- Setup Spring Boot project
- Configure PostgreSQL
- Setup Maven dependencies

Day 2 – Entities & Repositories

- Create JPA entities
- Create repositories

Day 3 – Authentication

- Implement JWT authentication
- Configure Spring Security

Day 4 – Communities

- APIs for community creation & listing

Day 5 – Posts

- Create post APIs
- Fetch posts by community

Day 6 – Voting

- Implement vote logic
- Ensure one vote per user

Day 7 – Comments

- Add comment APIs
- Link comments with posts



Week 2 – Frontend + Integration

Day 8 – Frontend Setup

- Setup Next.js + Tailwind
- Connect APIs

Day 9 – Community UI

- Community pages
- Listing UI

Day 10 – Posts UI

- Post creation
- Post listing

Day 11 – Voting + Comments UI

- Integrate vote API
- Comment system

Day 12 – Auth Integration

- Login/signup UI
- Store JWT

Day 13 – Testing

- Test APIs & frontend integration

Day 14 – Deployment

- Deploy backend & frontend
 - Write documentation
-



Essential Essentials

Security Considerations

- Use JWT for authentication
 - Encrypt passwords (BCrypt)
 - Protect endpoints with Spring Security
 - Validate all inputs
-



UI Guidelines

- Clean Reddit-like layout
 - Responsive design
 - Clear interaction buttons
 - Loading & empty states
-



Future Enhancements

- Nested comments
 - Notifications
 - Real-time updates (WebSockets)
 - Admin moderation
 - Search functionality
-



Learning Resources

Backend

- [Spring Boot Docs](#)
- [Spring Security Docs](#)
- [Hibernate Docs](#)

Frontend

- [Next.js Docs](#)
- [Tailwind CSS Docs](#)

Conclusion

This project demonstrates:

- Strong backend development using Java
- API design and database modeling
- Full-stack integration
- Production-level architecture

It's a **high-impact portfolio project** for:

- Java Backend Developer
- Full Stack Developer roles

Resource List – Reddit Clone (Java Tech Stack)

1. Full Project / System Design References

End-to-End Backend + Frontend Ideas

- Spring Boot + React Full Stack Project
 <https://www.youtube.com/watch?v=vtPkZShrvXQ>
- Reddit System Design (important for voting + scaling logic)


How to use:

- Don't copy fully
 - Understand **flow** + **architecture** + **API design**
-

2. Core Backend – Spring Boot

-  Spring Boot Official Docs
 <https://spring.io/projects/spring-boot>
-  Spring Boot Full Course (Beginner → Advanced)
 <https://www.youtube.com/watch?v=9SGDpanrc8U>

Focus on:

- REST APIs (@RestController)
 - Dependency Injection
 - Layered architecture (Controller → Service → Repository)
-

3. Authentication & Security (VERY IMPORTANT)

Spring Security

-  Spring Security Docs
 <https://spring.io/projects/spring-security>
-  JWT Authentication with Spring Boot
 https://www.youtube.com/watch?v=her_7pa0vrg

 Focus on:

- JWT token generation
 - Filters & authentication flow
 - Securing endpoints
-

4. ORM & Database (Critical Part)

Hibernate (JPA)

-  Hibernate Docs
 <https://hibernate.org/orm/documentation/>
 -  Hibernate + JPA Tutorial
 <https://www.youtube.com/watch?v=QJqNYhiHysM>
-

PostgreSQL

-  PostgreSQL Docs
 <https://www.postgresql.org/docs/>
-  PostgreSQL Crash Course
 <https://www.youtube.com/watch?v=qw--VYLpxG4>

 Focus on:

- Relationships (OneToMany, ManyToOne)
 - Indexing
 - Query optimization
-

5. Build Tool & Project Setup

Maven

-  Maven Docs
 <https://maven.apache.org/guides/>
 -  Maven Crash Course

-

6. REST API Design

- REST API Best Practices
 <https://restfulapi.net/>

 Focus on:

- Resource naming (`/api/posts`)
 - Status codes
 - Request/Response structure
-

7. Voting System Logic (Important Feature)

- Reddit Voting Design


 Key Concept:

- Store votes in separate table
 - Prevent duplicate votes per user
 - Calculate score efficiently
-

8. Comment System

- Building Comment System (conceptual)



💡 MVP Tip:

- Start with **flat comments**
- Add nesting later if needed

9. Frontend Integration (Next.js / React)

-  Next.js Docs
👉 <https://nextjs.org/docs>
-  Tailwind CSS Docs
👉 <https://tailwindcss.com/docs>
-  React + API Integration
👉 <https://www.youtube.com/watch?v=Ke90Tje7VS0>

💡 Focus on:

- Fetch API / Axios
- Handling JWT in frontend
- Protected routes

10. Testing & Debugging

- API Testing: Postman
👉 <https://www.postman.com/>
- Spring Boot Testing
👉 <https://spring.io/guides/gs/testing-web/>

11. Image Upload (Optional)

Cloudinary

-  Cloudinary Docs
👉 <https://cloudinary.com/documentation>
 -  Spring Boot + Cloudinary
👉 <https://www.youtube.com/watch?v=Kc2v7yA5n4g>
-

12. Deployment

Backend Deployment

- Render
👉 <https://render.com/docs>
 - Railway
👉 <https://docs.railway.app>
-

Frontend Deployment

- Vercel
👉 <https://vercel.com/docs>
-

13. UI/UX Inspiration

- Reddit
👉 <https://www.reddit.com>

💡 Study:

- Voting buttons
 - Comment layout
 - Feed structure
-

⚡ Smart Strategy to Use These Resources

💧 Don't Do This:

- ✗ Watch 10 tutorials
 - ✗ Learn everything first
-

☑ Do This Instead:

1. Learn basics of:
 - Spring Boot
 - JWT Auth
 - JPA (relations)
 2. Build **feature by feature**:
 - Auth → Communities → Posts → Votes → Comments
 3. Use:
 - Docs when stuck
 - YouTube for understanding
-

🧠 Pro Tips (Important)

- Keep backend **clean layered architecture**
- Use DTOs (don't expose entities directly)
- Handle errors properly (`@ControllerAdvice`)
- Test APIs before frontend integration
- Commit code daily